

Università degli Studi dell'Insubria  
Dipartimento di Scienze Teoriche e Applicate  
Laboratorio Interdisciplinare A - 2026

---

# **CineMax**

Sistema di gestione cinematografica

---

## **Manuale Tecnico**

Versione 2.0 – 12/06/2026

### **Autori**

- Bin Alessio (Matricola: 762387) - VA
- Modena Matteo (Matricola: 765099) - VA
- Baroncelli Luca (Matricola: 761582) - VA

# Indice

1. Introduzione
2. Architettura dell'applicazione
  - 2.1 Flusso di controllo
  - 2.2 Gestione dei ruoli
3. Struttura dei Package
4. Strutture Dati
  - 4.1 Gerarchia degli utenti
  - 4.2 Lista utenti e proiezioni
  - 4.3 Paginazione
  - 4.4 Stack degli stati
5. Scelte algoritmiche
  - 5.1 Ricerca delle proiezioni
  - 5.2 Cifratura delle password
  - 5.3 Autenticazione e registrazione
  - 5.4 Gestione dei form TUI
6. Formato dei file e gestione dei path
  - 6.1 File CSV degli utenti
  - 6.2 File CSV delle proiezioni
  - 6.3 File CSV delle prenotazioni
  - 6.4 Gestione dei path
7. Pattern e scelte progettuali
  - 7.1 State machine con stack
  - 7.2 Separazione responsabilità
  - 7.3 Costanti di configurazione
  - 7.4 Installazione lazy del file
8. Descrizione delle classi principali (JavaDoc)
9. Limiti della soluzione
10. Sitografia/Bibliografia

## 1. Introduzione

CineMax è un sistema di gestione cinematografica basato su interfaccia testuale (TUI ) sviluppato in linguaggio Java. L'applicazione permette di gestire proiezioni cinematografiche, prenotazioni di biglietti e utenti per cinema monosala da 200 posti attraverso un'interfaccia grafica testuale intuitiva.

Il sistema supporta quattro tipologie di utenti:

- Cliente Ospite: può navigare e cercare film senza registrazione
- Cliente Registrato: può effettuare prenotazioni e gestire tali prenotazioni
- Proiezionista: può inserire, modificare ed eliminare proiezioni
- Bigliettaio: può visualizzare la programmazione, cercare prenotazioni e gestire le prenotazioni

L'applicazione utilizza un'architettura State Machine basata su uno Stack LIFO (Last-In-First-Out) per la navigazione tra le schermate. Ogni azione dell'utente può effettuare un push (avanzamento), un pop (ritorno) o un clear (reset/logout) dello stato applicativo.

## 2. Architettura dell'applicazione

L'architettura segue uno schema a livelli ispirato al pattern MVC adattato ad un contesto TUI.

Lo schema architetturale è il seguente:

- TUI(View): CinemaxTUI.java / MenuManager
- Controller: package cinemax.controller
- Model: package cinemax.model
- Persistenza: package cinemax.util
- Entry point: CineMax.java

### 2.1 Flusso di controllo

La navigazione tra le schermate viene gestita da una macchina a stati finiti implementata con un'enum StatoMenu e uno Stack<StatoMenu>. Ad ogni iterazione dell'utente il metodo statoMenuSuccessivo() della classe LogicaStatiManager calcola il prossimo stato ed esegue un push sullo Stack, di conseguenza quando si torna indietro viene eseguito un pop.

Esempio navigazione in avanti:

```
CineMax.stackRecord.push(StatoMenu.VISUALIZZA_PROGRAMMAZIONE);
```

Esempio navigazione indietro;

```
CineMax.stackRecord.pop();
```

### 2.2 Gestione dei ruoli

Il ruolo dell'utente corrente è memorizzato in CineMax.ruolo, un oggetto di tipo Ruolo, un enum con un'etichetta testuale. I controller leggono il valore per determinare quali azioni sono disponibili all'utente e forniscono il percorso possibile attraverso blocchi switch e if su CineMax.ruolo.getEtichetta.

### 3. Struttura dei Package

Struttura dei sorgenti

cinemax/

- |— CineMax.java (entry point, stato globale)
- |— CinemaxTUI.java (rendering schermate TUI)
- |— LogicaStatiManager.java (gestione stati e form)
- |— MenuManager.java (enum StatoMenu con transizioni)
- |— CostantiForm.java (enum Campi con indici form)
- |— controller/
  - |— FilmController.java
  - |— PrenotazioniController.java
  - |— AutenticazioniController.java
- |— model/
  - |— Utente.java (classe astratta base)
  - |— Ruolo.java (enum)
  - |— Proiezionisti.java
  - |— Bigliettai.java
  - |— Clienti.java
  - |— ClientiRegistrati.java
  - |— ClientiOspiti.java
  - |— Proiezione.java
  - |— Prenotazione.java
- |— util/
  - |— GestoreUtenti.java
  - |— GestoreProiezione.java
  - |— GestorePrenotazione.java
  - |— Cifratura.java

data/

- |— utenti.csv    persistenza degli utenti
- |— proiezioni.csv    persistenza delle proiezioni
- |— prenotazioni.csv    persistenza delle prenotazioni

bin/

- |— Laboratorio\_A.jar    JAR eseguibile compilato

## 4. Strutture dati

### 4.1 Gerarchia degli utenti

La gerarchia delle classi che gestiscono l'utente segue il principio dell'ereditarietà di java:



La classe astratta Utente contiene i campi comuni: nome, cognome, username, password (cifrata), dataDiNascita, domicilio e ruolo. Le sottoclassi si occupano di impostare direttamente il ruolo con il costruttore della superclasse.

### 4.2 Lista utenti e proiezioni

La classe GestoreUtenti mantiene la lista di utenti in memoria come LinkedList. La scelta di LinkedList è appropriata per operazioni di inserimento e rimozione frequenti, a fronte di un accesso sequenziale durante ricerche e salvataggio.

GestoreProiezione mantiene invece le proiezioni in una ArrayList, privilegiando l'accesso per indice necessario alla paginazione.

### 4.3 Paginazione

La classe FilmController implementa una paginazione lato controller: le proiezioni trovate vengono memorizzate in proiezioniTrovate, e la pagina corrente viene calcolata moltiplicando paginaCorrente per ELEMENTI\_PAGINA, che è una costante impostata a 10. I flag esistenzaPaginaSuccessiva e esistenzaPrecedente abilitano la navigazione avanti/indietro nella TUI.

```
public static boolean esistenzaPaginaSuccessiva;
public static boolean esistenzaPrecedente;
public static int paginaCorrente = 0;
public static final int ELEMENTI_PAGINA = 10;

public static void aggiornaProiezioniPerPagina(){
    filmPaginaTmp.clear();
    esistenzaPaginaSuccessiva = false;
    esistenzaPrecedente = false;
    int indiceInizio = ELEMENTI_PAGINA * paginaCorrente;
    for(int i = indiceInizio; i < indiceInizio + ELEMENTI_PAGINA && i < proiezioniTrovate.size(); i++){
        filmPaginaTmp.add(proiezioniTrovate.get(i));
    }
    if(indiceInizio + ELEMENTI_PAGINA < proiezioniTrovate.size()) esistenzaPaginaSuccessiva = true;
    if(paginaCorrente > 0) esistenzaPrecedente = true;
}
```

### 4.4 Stack degli stati

CineMax.stackRecord è uno `Stack<StatoMenu>` che modella la cronologia degli stati di navigazione. Questo consente di tornare indietro di schermata con un `pop()` senza che siano i singoli metodi a tenere traccia dello stato precedente.

## **5. Scelte algoritmiche**

### **5.1 Ricerca delle proiezioni**

Il metodo `GestoreProiezione.cercaProiezione()` supporta la ricerca per titolo, intervallo di date, prezzo e genere. Tutti i parametri sono opzionali: se nulli o vuoti, il filtro non è applicato. La ricerca opera dall'inizio alla fine sull'intera lista di proiezioni in memoria, con complessità  $O(n)$ .

### **5.2 Cifratura delle password**

La classe `Cifratura` applica una funzione di hash alle password prima della loro memorizzazione nel file CSV. Il confronto in fase di login avviene tra la password inserita (re-hashata) e quella memorizzata, la password non viene mai salvata in chiaro così.

### **5.3 Autenticazione e registrazione**

Il metodo `GestoreUtenti.registraUtente()` verifica:

- la completezza dei campi obbligatori
- la corrispondenza tra password e conferma password
- l'unicità dello username

In caso di errore solleva `IllegalArgumentException` con messaggio descrittivo. Il metodo `login()` cifra la password inserita e la confronta con quella memorizzata tramite ricerca lineare sulla lista.

### **5.4 Gestione dei form TUI**

`LogicaStatiManager.prendiDatiForm()` utilizza sequenze di escape ANSI per posizionare il cursore in coordinate specifiche del terminale, sovrascrivendo i campi disegnati dalla TUI. I dati vengono salvati nell'array `campiForm[]` con indici definiti dall'enum `Campi` nella classe `CostantiForm`. Digitare “:q” in qualsiasi campo annulla l'operazione e torna allo stato precedente.

## 6. Formato dei file e gestione del path

### 6.1 File csv degli utenti

Path: data/utenti.csv

Nel file i dati vengono salvati secondo questo schema:

nome|cognome|username|password|dataDiNascita|domicilio|ruolo

Il separatore è il carattere |. La prima riga consiste nell'intestazione, difatti in fase di lettura questa riga viene saltata. dataDiNascita e domicilio possono essere salvati come null. Se all'avvio del programma il file non dovesse esistere verrebbe comunque creato dal programma con uno staff predefinito di 2 proiezionisti e 5 biglietti.

### 6.2 File csv delle proiezioni

Path: data/proiezioni.csv

Nel file i dati vengono salvati secondo questo schema:

data\_proiezione|ora\_proiezione|titolo\_film|genere|regista|anno|durata\_minuti|eta\_minima|prezzo|posti\_sala

Il separatore è il carattere |. Tutti i campi sono obbligatori.

### 6.3 File csv delle prenotazioni

Path: data/prenotazioni.csv

Nel file i dati vengono salvati secondo questo schema:

codice\_prenotazione|nome\_cliente|cognome\_cliente|username\_cliente|data\_proiezione|ora\_proiezione|titolo\_film|numero\_biglietti

Il separatore è il carattere |. Il codice\_prenotazione è generato come stringa esadecimale univoca (es. 3BEFB835)

### 6.4 Gestione dei path

I path sono relativi alla directory di lavoro corrente al momento dell'avvio del JAR. Il programma deve quindi essere avviato dalla cartella radice del progetto, come indicato nel README:

```
Per Eseguire:  
Per avviare CineMax, aprire un terminale o prompt dei comandi, spostarsi nella cartella principale del progetto, se ad esempio la vostra  
cartella si chiama LaboratorioA, il comando sarà: cd C:\percorso_file\LaboratorioA  
Per eseguire il programma digitare a terminale: java -jar bin/Laboratorio_A.jar
```

## **7. Pattern e scelte progettuali**

### **7.1 State machine con stack**

La navigazione dell'applicazione è modellata come una macchina a stati. L'enum StatoMenu contiene tutti gli stati possibili, ciascuno dei quali ha a disposizione un metodo prossimi() che restituisce gli stati raggiungibili da quello attuale. Questo separa la definizione della topologia di navigazione (dentro l'enum) dalla logica di gestione degli input (dentro i controller).

### **7.2 Separazione responsabilità**

Le classi Gestore\* nel package util si occupano esclusivamente della persistenza su file (lettura/scrittura CSV), mentre i controller gestiscono la logica applicativa e il model rappresenta i dati di dominio. Questa separazione rende ciascun componente sostituibile indipendentemente.

### **7.3 Costanti di configurazione**

L'enum Campi in CostantiForm.java centralizza tutti gli indici degli array dei form, evitando numeri errati sparsi nel codice e rendendo immediata la corrispondenza tra campo grafico e slot dell'array datiFormTmp[].

### **7.4 Inizializzazione lazy del file**

Se i file CSV non esistono all'avvio, i Gestore\* li creano con dati predefiniti (metodo inizializzaStaffDefault()). Questo elimina la necessità di uno step di configurazione iniziale manuale.

## 8. Descrizione delle classi principali

### Package cinemax

#### CineMax

Classe principale (entry point). Contiene lo stato globale dell'applicazione: stackRecord (Stack), ruolo (Ruolo dell'utente corrente) e utenteCorrente (Utente loggato). Avvia il loop principale che cicla sullo stack degli stati.

#### LogicaStatiManager

Gestisce la logica di cambiamento degli stati del menu e la raccolta dell'input utente nei form. Il metodo statoMenuSuccessivo(String indice) calcola la transizione in base alla scelta numerica, mentre prendiDatiForm(String[], String[]) gestisce l'input contestualizzato alle coordinate ANSI della TUI.

#### MenuManager (enum StatoMenu)

Definisce tutti gli stati dell'applicazione e le rispettive transizioni attraverso il metodo prossimi(). Ogni stato conosce i propri successori, separando la struttura di navigazione dalla logica di controllo.

### Package cinemax.model

#### Utente (classe astratta)

Modella un utente del sistema con attributi: nome, cognome, username, password (cifrata), dataDiNascita, domicilio, ruolo. Le sottoclassi Proiezionisti, Bigliettai e ClientiRegistrati impostano automaticamente il ruolo nel costruttore.

#### Proiezione

Rappresenta una singola proiezione cinematografica con i campi: data (LocalDate), ora (String), titolo, genere, regista, anno, durata, etaMin, prezzo, postiSala. Espone getter e setter per tutti i campi.

Ruolo (enum) Definisce i quattro ruoli del sistema: PROIEZIONISTA, BIGLIETTAIO, CLIENTE\_REGISTRATO, OSPITE. Ciascun valore espone un'etichetta testuale tramite getEtichetta(), utilizzata nei confronti nei controller.

### Package cinemax.controller

#### FilmController

Gestisce la logica di business relativa alle proiezioni: ricerca (gestisciCercaFlm), paginazione (aggiornaProezioniPerPagina), selezione, inserimento (gestisciInserimentoProiezione) e modifica (gestisciModificaProiezione). Il comportamento al momento della selezione di un film dipende dal ruolo dell'utente.

#### PrenotazioniController

Gestisce la logica di prenotazione e acquisto biglietti, incluso il controllo della disponibilità dei posti.

### AutenticazioniController

Gestisce il login, la registrazione e il logout. Delega la persistenza a GestoreUtenti e aggiorna lo stato globale in CineMax.

### Package cinemax.util

#### GestoreUtenti

Si occupa di leggere e scrivere il file data/utenti.csv. Mette a disposizione metodi statici: caricaUtenti(), salvaUtenti(), login(String, String), registraUtente(...), cercaPerUsername(String). In assenza del file CSV genera automaticamente lo staff predefinito.

#### GestoreProiezione

Gestisce la persistenza del file data/proiezioni.csv. Mette a disposizione: leggiProiezioni(), salvaSuFile(), cercaProiezione(...), inserisci(Proiezione), elimina(Proiezione).

#### GestorePrenotazione

Gestisce la persistenza del file data/prenotazioni.csv con operazioni di inserimento e lettura delle prenotazioni.

#### Cifratura

Fornisce il metodo statico cifra(String) che applica un algoritmo di hash alla stringa passata come parametro. Utilizzato per la cifratura delle password prima della persistenza e per la verifica in fase di login.

## 9. Limiti della Soluzione Sviluppata

La versione attuale di CineMax presenta alcune limitazioni note, elencate di seguito:

- Interfaccia testuale: il programma non dispone di un'interfaccia grafica (GUI). Tutta l'interazione avviene tramite testo nel terminale, utilizza codice ANSI per renderizzare la pagina attuale. L'interfaccia è ottimizzata per terminali di dimensione 135x45 caratteri.
- Nessun controllo date passate: il sistema non impedisce la prenotazione di proiezioni con data già trascorsa.
- Nessuna persistenza in tempo reale: le modifiche vengono salvate su file CSV; eventuali interruzioni improvvise del programma potrebbero causare perdita di dati.
- Accesso multi-utente non supportato: il sistema è progettato per un singolo utente alla volta.
- Funzionalità non implementate: recupero password/username
- Validazione Input: controlli di formato input basilari
- Stato programma: se si vuole vedere lo stato attuale del programma, andare nel file CinemaxTUI.java:

```
public static void renderizzaMenu(StatoMenu statoMenu){
    clearConsole();
    System.out.println(bordoSuperiore);
    System.out.println(rigaVuota);
    formattaIntestazione(statoMenu.getNomeLogo());
    System.out.println(rigaVuota);
    System.out.println(bordoMezzo);
    System.out.println(rigaVuota);
    System.out.println(rigaVuota);
    if(statoMenu == StatoMenu.STATO_ERRORE){
        formattaTesto(LogicaStatiManager.messaggioErroreCorrente.toUpperCase(), posizione: "centro", visualizzaNumeri: true);
    } else if(statoMenu == StatoMenu.STATO_CONFERMA){
        formattaTesto(LogicaStatiManager.messaggioConfermaCorrente.toUpperCase(), posizione: "centro", visualizzaNumeri: true);
    } else if(statoMenu == StatoMenu.VISUALIZZA_PROGRAMMAZIONE || statoMenu == StatoMenu.MIE_PRENOTAZIONI){
        stampaListaPagina();
    } else if(statoMenu == StatoMenu.MENU_INFO_FILM){
        gestisciInformazioniFilm(cinemax.controller.FilmController.filmSelezionatoTmp);
    } else if(statoMenu == StatoMenu.DETTAGLIO_PRENOTAZIONE){
        gestisciDettaglioPrenotazione(cinemax.controller.PrenotazioniController.prenotazioneSelezionataTmp);
    } else
        formattaTesto(statoMenu.getOpzioni(), statoMenu.getPosizione(), statoMenu.getVisualizzaNumeri());

    System.out.println(rigaVuota);
    System.out.println(rigaVuota);
    System.out.println(bordoInferiore);
    if(CineMax.stackRecord.peek().getVisualizzaNumeri())System.out.print(s: "\nSELEZIONA UN'OPZIONE: ");
    //System.out.println(CineMax.stackRecord);
}
```

Nel metodo renderizzaMenu() andare a rimuovere le // dell'ultima riga.

- Capacità del Sistema: Prestazioni possono degradare con migliaia di proiezioni

## 10. Bibliografia e Sitografia

### Documentazione Tecnica

1. [Oracle Java Documentation](#)

- Consultato per: API Java, gestione I/O, strutture dati

2. [Java Tutorials - Oracle](#)

- Consultato per: Best practices, pattern di programmazione

### Standard e Specifiche

1. [ANSI Escape Codes](#)

- Consultato per: Sequenze di escape per controllo cursore e formattazione testuale

2. [Unicode Standard](#)

- Consultato per: Caratteri box-drawing e simboli grafici

### Materiale Didattico

1. Documentazione dei Corsi

## Note Finali

Per assistenza tecnica o segnalazione bug, contattare gli sviluppatori:

- Bin Alessio - [abin@studenti.uninsubria.it](mailto:abin@studenti.uninsubria.it)
- Modena Matteo - [mmodena@studenti.uninsubria.it](mailto:mmodena@studenti.uninsubria.it)
- Baroncelli Luca - [lbaroncelli@studenti.uninsubria.it](mailto:lbaroncelli@studenti.uninsubria.it)